

current-state

Flatcar Security/CVE Handling — Current State (As-Is)

Compiled from: direct inspection of `flatcar/scripts`, `flatcar/Flatcar`, `flatcar/jenkins-os`, `flatcar/flatcar-build-scripts`, live GitHub issue/PR queries, the real `security.gentoo.org` RSS feed, and a Slack conversation with **Dongsu Park** (2026-07-17). Every fact below was verified against source (file paths and commands are cited in §7); nothing here is speculative.

Scope note: this document describes **what exists today**. It is deliberately *not* a proposal — the target VEX design is a separate follow-up document.

Contents: [1. Executive Summary](#) · [2. Overall Map](#) · [3. Path A — GLSA Build Gate](#) · [4. Path B — Routine Updates](#) · [5. Path C — Manual Advisory Tracking](#) · [6. Data Sources Inventory](#) · [7. Known Gaps](#) · [8. Verified Facts Reference](#)

1. Executive Summary

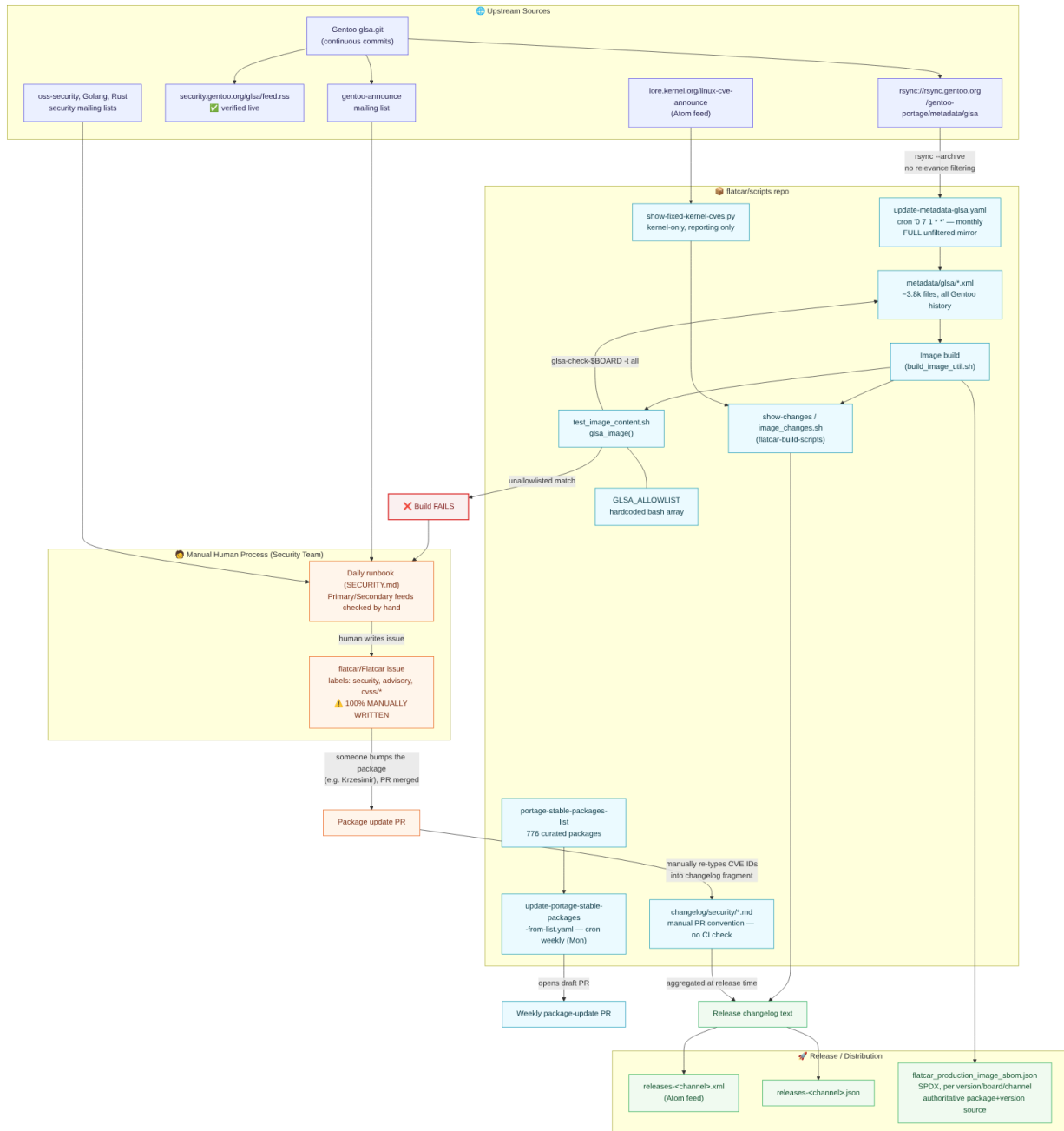
There are **two largely decoupled mechanisms** by which vulnerabilities get addressed in Flatcar today, plus a **third, purely manual** communication layer sitting on top of both. There is currently **no machine-readable VEX output anywhere** in this system.

#	Mechanism	Covers	Automated?	Blocks a build?
A	<code>glsa-check</code> build gate	Only CVEs Gentoo wrote a formal GLSA for	Detection:  · Triage: 	 Yes
B	Routine / weekly package updates	The majority of real-world CVE fixes	Sync:  · Review/merge: 	 No

#	Mechanism	Covers	Automated?	Blocks a build?
C	Manual advisory issues (flatcar/Flatcar)	Human-curated superset, independent of A/B	✗ 100% manual	✗ No

Key correction from Dongsu Park: A and B are **largely independent**. Most CVEs are fixed as a *side effect* of ordinary package-freshness bumps (Path B) — not because a GLSA ever fired. GLSA/glsa-check (Path A) only ever sees “a limited list of CVEs.”

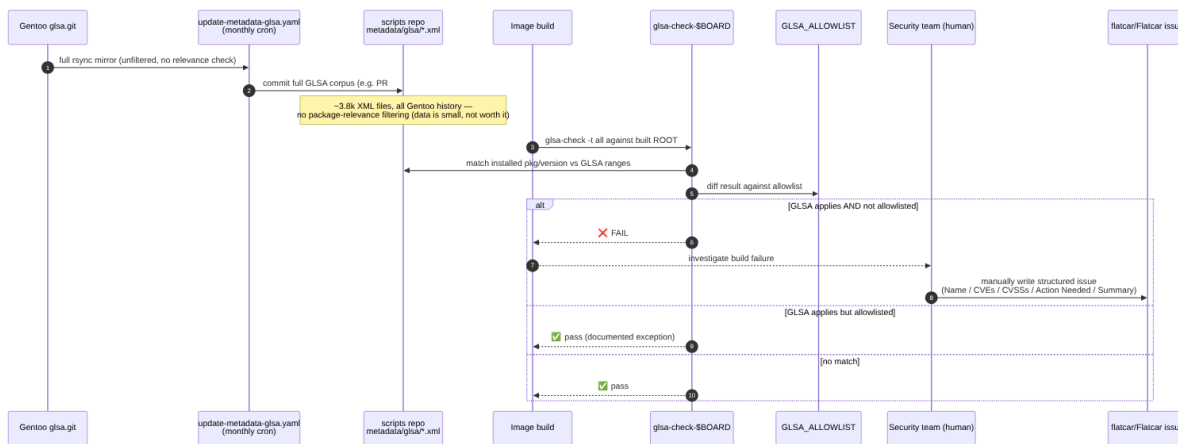
2. Overall Map



diagram

3. Path A — GLSA Build-Gate Lifecycle

This is the **only fully automated *detection*** mechanism in the current system — but it only fires for the subset of CVEs that have a formal Gentoo GLSA, and only at **build time**, against whatever is currently being built (not already-shipped releases).



diagram

Source: `build_library/test_image_content.sh` → `glsa_image()`, invoked from `build_image_util.sh` (`test_image_content()`).

What `glsa-check` actually is (provenance)

`glsa-check` is **not Flatcar-authored tooling** — it ships as part of upstream `sys-apps/portage` itself (Gentoo's package manager; confirmed via `gentoo/portage` source: `bin/glsa-check` + `lib/portage/glsa.py`), which Flatcar vendors unmodified in `portage-stable` (`sys-apps/portage/portage-3.0.81.ebuild`) as a hard SDK dependency. `glsa-check-$BOARD` is not a distinct per-board tool — it's Portage's standard board-scoped binary naming convention (same as `emerge-$BOARD`), same underlying script. Per the official man page (`man/glsa-check.1`), Flatcar's `-t/--test` invocation is the “detect only” mode; `glsa-check` also supports `-f/--fix` (experimental auto-remediation) and `-i/--inject` (permanent per-GLSA ignore list) — Flatcar uses neither, implementing its own equivalent of `-i` via the hand-maintained `GLSA_ALLOWLIST` bash array instead.

The actual matching algorithm (`lib/portage/glsa.py`, traced directly from source)

Each GLSA XML's `<affected><package>` block declares per-arch `<vulnerable>/<unaffected>` version nodes with a `range` attribute. A real example from Flatcar's vendored corpus (`200310-03`):

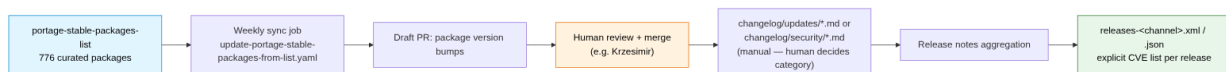
```
<affected>
  <package name="www-servers/apache" auto="yes" arch="*">
    <unaffected range="ge">1.3.29</unaffected>
    <vulnerable range="lt">1.3.29</vulnerable>
  </package>
</affected>
```

1. `range` maps to a Portage atom operator via a fixed table: `le→<=`, `lt→<`, `eq→=`, `gt→>`, `ge→>=`, plus revision-qualified `rge/rle/rgt/rlt → >=~/<=~/>~/<~` (Portage’s “same version, compare only the `-rN` revision suffix” tilde syntax).
2. Each node becomes a real atom string (e.g. `<www-servers/apache-1.3.29`), optionally slot-qualified.
3. `isVulnerable()` walks each `<package>` block, filters by `arch` (* or a space-separated list — the built image’s arch must be in it), then matches each vulnerable atom against the **installed vardb** (`vardbapi` — whatever’s actually merged into `ROOT`), subtracting anything that also matches an `<unaffected>` atom.
4. **Availability of a fix does not gate the affected/not-affected verdict** — `getMinUpgrade()` (which computes whether an upgrade path exists in the tree) returns an empty list, not `None`, when the system is vulnerable but no fix is available yet; since `isVulnerable()` only checks `is not None`, the verdict is purely a **version-range match against installed package state**. The upgrade-path computation is used solely by `-p/-pretend`’s remediation-guidance output, a separate code path.

Relevance to a future Go VEX matcher: the core algorithm — atom-operator lookup + version-range containment + arch filter — has no live-system dependency beyond *which* database it matches against. For build-time gating that’s the built image’s vdb; for a retrospective per-release VEX generator it would instead be the SBOM’s recorded package+version, playing the same role `vardbapi` plays here. The version-comparison semantics (including the `~/revision-qualified` operators) should be ported faithfully rather than reimplemented as naive semver, since Portage version comparison has real edge cases a semver-style comparator would get wrong.

4. Path B — Routine / Weekly Package Updates (Independent of GLSA)

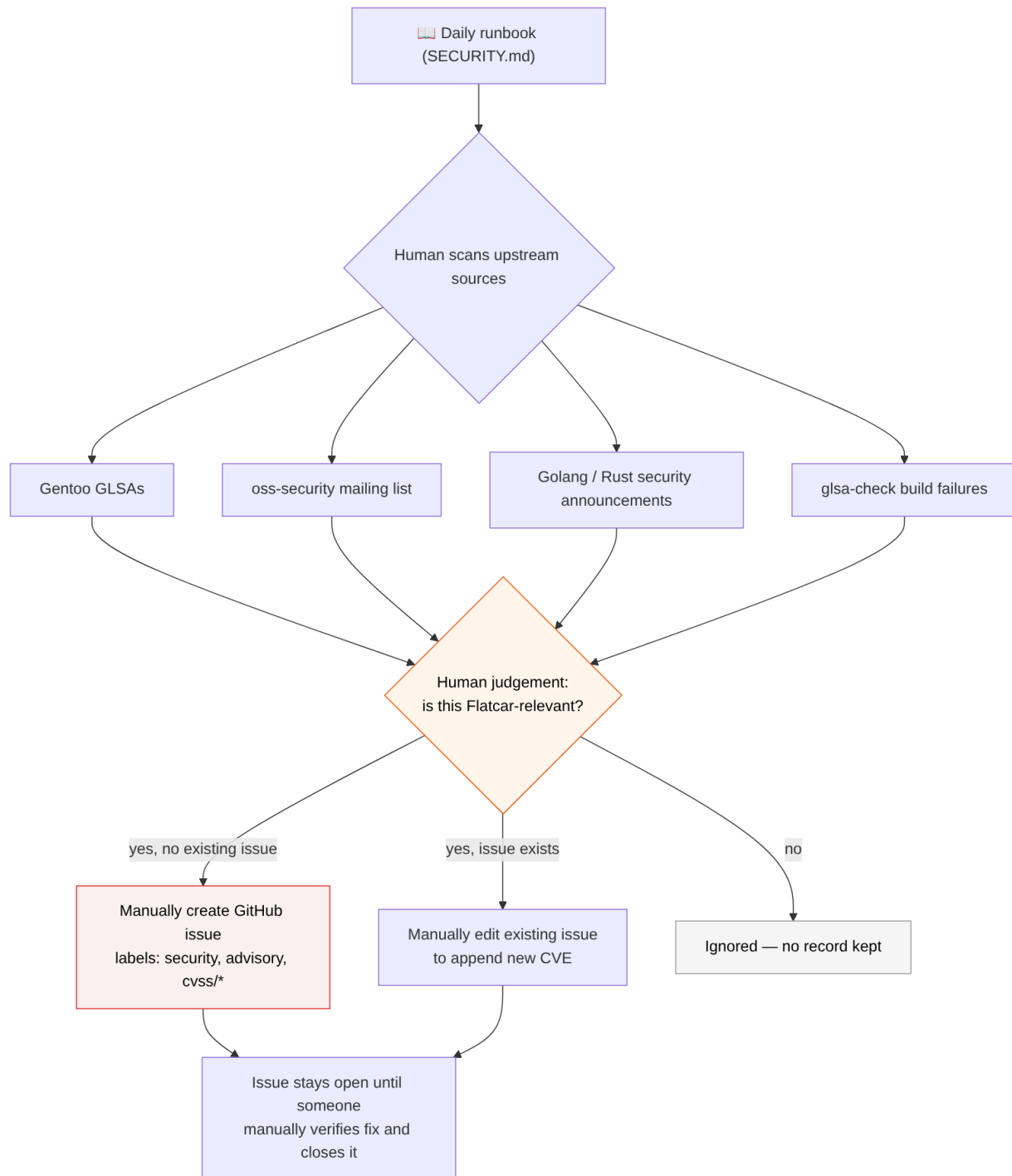
Per Dongsu: **most real CVE fixes happen here**, not via Path A. A package gets bumped for ordinary freshness reasons; if that bump happens to cross a CVE-fixed version boundary, the CVE is fixed as a side effect — with no GLSA or `glsa-check` failure ever involved.



diagram

5. Path C — Manual Advisory Issue Tracking

Confirmed directly by Dongsu: **“Those are all manually created”** — referring to the advisory-labeled issues in flatcar/Flatcar (**335 total, 50 currently open** as of this review — see §8).



diagram

Observed issue body template (consistent across years/authors, still 100% hand-typed):

Name: <package>

CVEs: <CVE-1>, <CVE-2>, ...

CVSSs: <score-1>, <score-2>, ...

Action Needed: update to >= <version>

Summary: <upstream description>

Severity labels — `cvss/CRITICAL` (≥ 9), `cvss/HIGH` (7–9), `cvss/MEDIUM` (4–7) — are also assigned by hand, consistently enough to look templated, but no code anywhere produces them. A single issue (e.g. [#2188](#)) can bundle **multiple CVEs with different Action Needed thresholds** — an important edge case for any future automation.

6. Data Sources Inventory



Detection / Input Sources

Gentoo GLSA rsync mirror
(full corpus, monthly)

security.gentoo.org/glsa/feed.rss
near real-time, verified working

gentoo-announce
mailing list

[lore.kernel.org/linux-cve-
announce](https://lore.kernel.org/linux-cve-announce/)
(kernel-only)

oss-security / Golang / Rust
mailing lists — human-read
only



✓ Ground Truth for 'what do we ship'

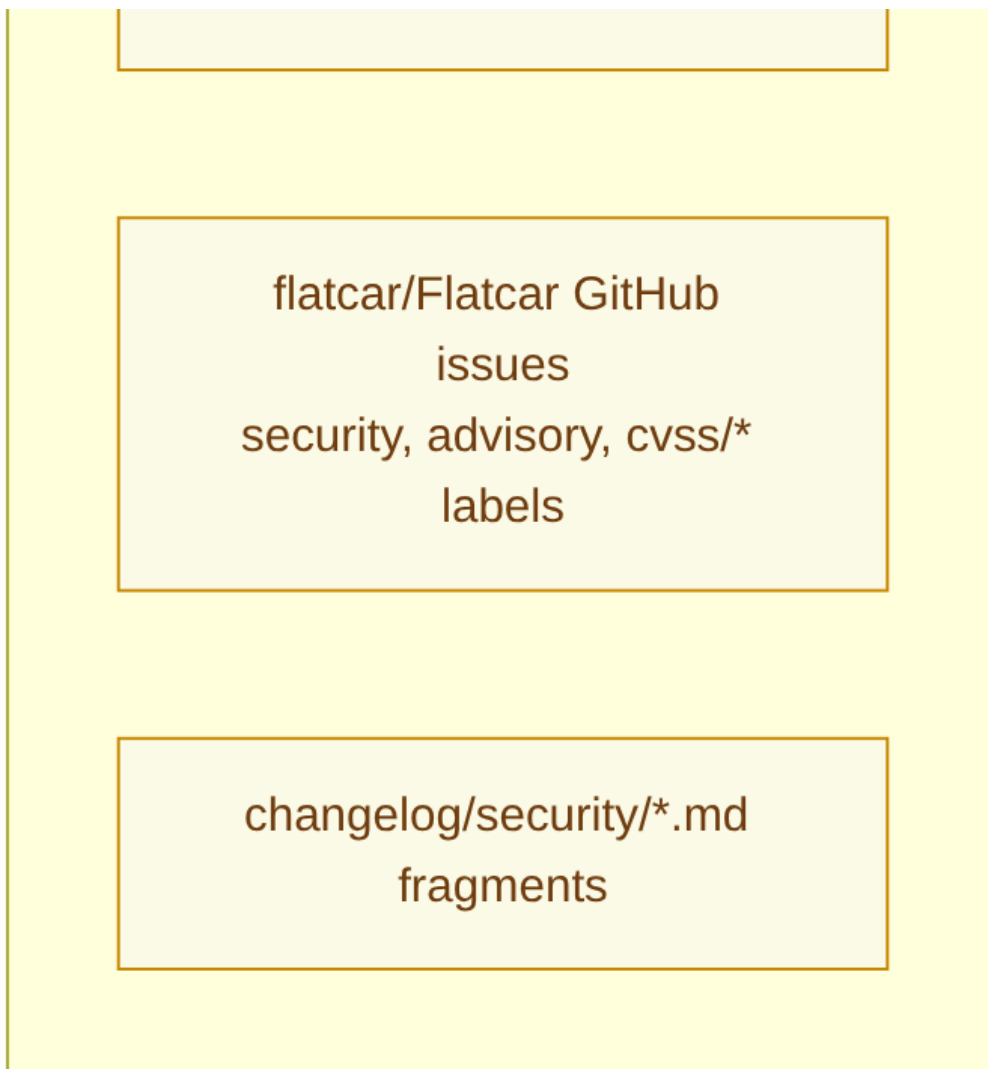
portage-stable + coreos-
overlay
≈599 packages (446 + 153)
curated subset of Gentoo's
~19,000

flatcar_production_image_sbom.json
SPDX; 'ebuild' purl type =
exact category/pkg + version



Published Outputs

releases-
<channel>.xml/.json
explicit fixed-CVE lists per
release



diagram

G1 is a reliable **existence proxy** (“is this package used by Flatcar at all”) but not sufficient alone to prove something is in the *production-shipped* image (it could be SDK-only or sysext-only). G2 (the SBOM) is authoritative for that finer distinction and is also the only source that ties an exact **version** to a **released** artifact.

Confirmed absence of any tooling for non-ebuild packages: the SBOM’s `golang` purls (~1070 of ~1400 total packages) and any Rust/crates.io packages have **no automated vulnerability-detection mechanism of any kind** today — not GLSA (Gentoo doesn’t write GLSAs for most Go modules), and not a generic scanner either. Directly checked: no `dependabot.yml`, and no `govulncheck/cargo-audit/trivy/grype/snyk` workflow exists anywhere in `flatcar/Flatcar`, `flatcar/scripts`, or `flatcar/jenkins-os` (all `.github/workflows/*.yml` files enumerated and searched directly). These packages only

receive security fixes as a side effect of Path B’s routine freshness bumps, with zero CVE-awareness built into that process — this is a genuine blind spot, not merely “less structured” than GLSA coverage.

7. Known Gaps in the Current State

#	Gap	Why it matters
1	No automated relevance filtering	A human must eyeball every new GLSA/CVE against Flatcar’s package list; nothing does this today.
2	GLSA coverage is narrow	Most real fixes happen via Path B, invisible to <code>glsa-check</code> entirely, since A and B are decoupled.
3	Advisory issue creation is 100% manual	<code>SECURITY.md</code> claims “we... use automation tools to create a new issue” — confirmed false by Dongsu.
4	CVE IDs re-typed by hand, twice	Once into the GitHub issue, again into <code>changelog/security/*.md</code> — no automated link, no CI enforcement that the fragment is even added.
5	No VEX output anywhere	Nothing here produces a machine-readable VEX document; the closest analogues are hand-written issue bodies and release-note CVE lists.
6	No queryable “shipped version X ↔ still-affected-by CVE Y” record	That information lives only in prose (issue comments like “Fixed in Beta 4722.1.0, Stable 4593.2.4, LTS 4081.3.9”).
7	Zero automated CVE detection for non-ebuild (Go/Rust) packages	~1070 of ~1400 SBOM packages are golang purls with no GLSA and no other scanner (<code>dependabot/govulncheck/cargo-audit/trivy/grype/snyk</code> all confirmed absent repo-wide); these only get fixed as a side effect of routine freshness bumps (Path B), with no CVE-awareness at all.

8. Verified Facts Reference

Every quantitative claim above was checked against a primary source on 2026-07-17, with the advisory-issue count re-verified on 2026-07-21 (counts drift daily; the rest were unchanged):

Claim	Verified value	Source
GLSA sync cadence	<code>cron: '0 7 1 * *'</code> (monthly, 1st)	<code>scripts/.github/workflows/update-metadata-glsa.yaml</code>
Weekly package sync cadence	<code>cron: '0 7 * * 1'</code> (weekly, Monday)	<code>scripts/.github/workflows/update-portage-stable-packages-from-list.yaml</code>
Local GLSA XML mirror size	3,817 files	<code>find scripts -iname 'glsa-*.xml' \ wc -l</code>
Curated package list size	776 entries	<code>scripts/.github/workflows/portage-stable-packages-list</code>
Package tree size (ground truth)	446 (portage-stable) + 153 (coreos-overlay) = 599	GitHub API tree listing, flatcar-archive/portage-stable & flatcar-archive/coreos-overlay @ main
Build gate function	<code>glsa_image() → glsa-check-\$BOARD -t all</code> vs <code>GLSA_ALLOWLIST</code>	<code>scripts/build_library/test_image_content.sh</code>
advisory-labeled issues	335 total, 50 open	GitHub Search API, repo:flatcar/Flatcar label:advisory (re-checked 2026-07-21)
“Automation” claim in docs	Present, contradicted by Dongsu	Flatcar/SECURITY.md line 28
GLSA RSS feed	Live and working	https://security.gentoo.org/glsa/feed.rss
GLSA notification mailing list	<code>gentoo-announce</code>	Gentoo project documentation
<code>glsa-check</code> provenance	Ships with <code>sys-apps/portage</code> , not <code>gentoolkit</code> or Flatcar-authored	gentoo/portage source (bin/glsa-check, lib/portage/glsa.py, man/glsa-check.1); corroborated by Gentoo Wiki’s “GLSA” page (“glsa-check application (distributed with Portage)”)
<code>glsa-check</code> matching semantics	Version-range match against installed vdb only; fix-availability does not gate the affected verdict	gentoo/portage lib/portage/glsa.py (isVulnerable(), getMinUpgrade()), re-checked 2026-07-21
Non-ebuild (Go/Rust) vulnerability tooling	Confirmed absent — no <code>dependabot.yml</code> , no <code>govulncheck/cargo-audit/trivy/grype/snyk</code> workflow	Direct enumeration + search of all <code>.github/workflows/*.yml</code> in flatcar/Flatcar, flatcar/scripts, flatcar/jenkins-os, 2026-07-21